

## FICHE DE TD 3/3 • DIFFICILE

## Informatique • 1re C D E

## Programmation en JavaScript

## Module 2 : Algorithmique et programmation

Cette fiche d'évaluation synthétique mobilise l'ensemble des notions du module : traduction algorithmique, typage dynamique, interaction DOM événementielle et conception modulaire.

**EXERCICE 1** Transcodage algorithmique et pièges du typage

On souhaite implémenter en JavaScript l'algorithme suivant calculant la somme  $S$  des  $N$  premiers termes de la suite définie par  $u_0 = 1$  et  $u_{n+1} = 2u_n + (-1)^n$ .

Variables :  $N, i, u, S$  : Entiers

Début

Afficher "Entrez N (entier > 0) : "

Saisir  $N$

$u \leftarrow 1$

$S \leftarrow 0$

Pour  $i$  de 0 à  $N-1$  faire

$S \leftarrow S + u$

$u \leftarrow 2 * u + (-1)^i$

FinPour

Afficher "Somme  $S =$ ",  $S$

Fin

1. Proposer une implémentation JavaScript complète (balise `<script>`) utilisant `prompt()` pour la saisie et `alert()` pour l'affichage. Gérer explicitement la conversion de type.
2. L'utilisateur saisit la chaîne "10" (avec guillemets) ou "5.7". Analyser le comportement de votre code face à ces entrées : que renvoie `parseInt()` ? `parseFloat()` ? `Number()` ? Justifier le choix le plus robuste pour  $N$ .
3. Réécrire la boucle Pour avec une structure `while` équivalente. Démontrer par récurrence sur  $k$  que la variable  $u$  vaut bien  $u_k$  à l'entrée de l'itération  $k$ .
4. Critiquer l'utilisation de `document.write()` pour afficher le résultat final dans une page HTML moderne déjà chargée. Proposer une alternative standard manipulant le DOM (sans écrire le code complet).

**EXERCICE 2** Programmation événementielle : Convertisseur multi-devises

On veut concevoir une interface web permettant de convertir un montant en Francs

CFA (XAF) vers Euro (EUR) ou Dollar US (USD). Taux fixes : 1 EUR = 655,957 XAF ; 1 USD = 600 XAF. L'interface HTML propose : un `<input type="number" id="montant">`, un `<select id="devise">` (options EUR/USD), un `<button id="btn">Convertir</button>` et un `<div id="resultat">`.

1. Écrire la fonction JavaScript `convertir()` appelée par l'événement `onclick` du bouton. Elle doit : récupérer les valeurs, valider que le montant est un nombre positif fini, calculer le résultat arrondi à 2 décimales, et l'injecter dans `#resultat` sous la forme "Montant : X.XX DEVISE". Gérer le cas d'erreur par un message rouge.
2. Expliquer pourquoi `parseFloat(document.getElementById('montant').value)` est préférable à `parseInt(...)` ici. Que se passe-t-il si l'utilisateur saisit "1000abc" ?
3. Modifier l'algorithme pour intégrer une commission bancaire de 1,5% prélevée sur le montant XAF avant conversion. Écrire l'expression mathématique exacte du résultat final  $R$  en fonction du montant saisi  $M$  et du taux  $T$ , puis la traduction JavaScript avec les opérateurs `*` et `/`.
4. Question ouverte : L'API `prompt()` est bloquante (modale). Comparer l'expérience utilisateur (UX) entre une saisie via `prompt()` et une saisie via `<input>` + événement `onclick`. Citer deux avantages majeurs de la seconde approche pour une application réelle.

### EXERCICE 3 Analyse critique de code : Portée, coercition et boucles

On donne le fragment de code suivant, inséré dans une page HTML contenant un bouton `<button id="test">Test</button>` et un paragraphe `<p id="log"></p>`.

```
<script>
  var total = 0;
  function ajouter(valeur) {
    total += valeur;
    return total;
  }
  document.getElementById('test').onclick = function() {
    var total = 10; // Ligne A
    for (var i = 0; i < 3; i++) {
      setTimeout(function() {
        var msg = "i=" + i + " total=" + ajouter(i); // Ligne B
        document.getElementById('log').innerHTML += msg + "<br>";
      }, 100);
    }
    var i = 20; // Ligne C
  };
</script>
```

1. Prédire l'affichage exact dans `#log` après un clic unique sur le bouton. Justifier précisément la valeur de `i` dans la fermeture (closure) du `setTimeout` (liaison avec `var` vs `let`).
2. Analyser l'impact de la Ligne A sur la variable globale `total` et sur le résultat de `ajouter()`. Que vaut `total` (global) à la fin de l'exécution?
3. Remplacer `var i` par `let i` dans la boucle `for`. Expliquer la différence de comportement sur l'affichage de `i` et sur la capture de la variable dans le callback.
4. La fonction `ajouter` utilise l'opérateur `+=`. Si on appelle `ajouter("5")` (chaîne), que se passe-t-il? Illustrer le risque de coercition implicite en JavaScript et proposer une garde défensive (type `guard clause`) au début de la fonction.

#### EXERCICE 4 Conception modulaire : Convertisseur de températures étendu

On doit réaliser un module JS réutilisable `TemperatureConverter` (objet littéral ou IIFE) exposant les méthodes `celsiusVersFahrenheit(c)`, `fahrenheitVersCelsius(f)`, `celsiusVersKelvin(c)`. Formules :  $F = C \times 9/5 + 32$ ,  $K = C + 273,15$ .

1. Écrire la structure de ce module en encapsulant les constantes (zéro absolu, rapports) en variables privées (non accessibles depuis l'extérieur). Implémenter les trois méthodes publiques avec validation d'entrée (nombre fini, respect du zéro absolu :  $C \geq -273,15$ ). Lever une `Error` explicite en cas d'erreur.
2. Dans le script principal, lier ce module à une interface HTML (trois `<input>` synchronisés : modification de l'un met à jour les deux autres en temps réel via l'événement `input`). Décrire l'algorithme de synchronisation évitant les boucles infinies de mise à jour.
3. Analyse de complexité : Si l'on ajoute une méthode `convertir(valeur, uniteSource, uniteCible)` générique utilisant une table de correspondance (objet JS) pour choisir la formule, quelle est la complexité temporelle de l'appel? Justifier.
4. Éthique/Robustesse : L'API `alert()` est souvent bloquée par les navigateurs. Proposer une stratégie d'affichage des erreurs de validation (zéro absolu violé, saisie non numérique) non intrusive, accessible (ARIA), et compatible avec le module conçu en question 1.